

Data Structures and Algorithms -23CSE203

Lab Manual

Submitted By

NAME	T.Naga Siddhardha
ROLL NO	AV.SC.U4CSE25151
YEAR/SEM/SECTION	2nd YEAR/ SEM-3/CSE-B
CREDITS	05

Submitted to

NAME	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Index

Sno	Title/Topics	Page No.	Date

1.	**WEEK 1:** **Recursion** A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	4-11	14-07-26
2.	**WEEK 2:** **Searching** A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	12-19	21-07-26
3.	**WEEK 3:** **Elementary Sorting** A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	20-25	01-08-26
4.	**WEEK 4:** **Advanced Sorting** A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).	26-30	11-08-26
5.	**WEEK 5:** **Linked Lists** Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL) Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit	31-38	18-08-26
6.	**WEEK 6:** **Stacks** A) Implement Stack operations (Push, Pop, Peek, Display) using Array. B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.	39-44	25-08-26

7.	**WEEK 7:** **Queues** A) Implement Queue using Arrays. B) Implement Queue using Linked List. C) Implement Circular Queue using Arrays. D) Implement Circular Queue using Linked List.	45-60	22-09-26
8.	**WEEK 8:** **Binary Trees** A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	61-68	29-09-26
9.	**WEEK 9:** **Binary Search Trees** Python program to implement Binary Search Tree (BST) operations.	69-75	06-10-26
10.	**WEEK 10:** **AVL Tree** Write a Python program to implement an AVL Tree with all its operations.	76-80	13-10-26
11.	**WEEK 11:** **Hashing** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	81-88	27-10-26

Week-1

a) A rocket launch system displays a count down before launch. Instead of using a loop, implement the count down using recursion. Write a recursive python program to display countdown from n to 1 followed by launch.

Input:

```
#T.Naga Siddhardha
def countdown(n):
    if n>0:
        print(n)
        countdown(n-1)
    else:
        print("Launch!")
count=5
countdown(count)
```

OUTPUT:

```
5
4
3
2
1
Launch!
```

Explanation:

The program uses **recursion** to display the countdown from n to 1. The function calls itself with n-1 until n becomes 0. After reaching the base condition, it displays **"Launch!"**. This performs the countdown without using a loop.

b) Banks calculate the growth of investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n where P = Principal growth factor, n = number of years

Input:

```
#T.Naga Siddhardha
def power(P,n):
    if n==0:
        return
    return P*power(P,n-1)
P=float(input("Enter P:"))
n=int(input("Enter n:"))
result=power(P,n)
print("P^n=",result)
```

Output:

```
Enter P: 7
Enter n: 5
P^n = 16807.0
```

Explanation:

The recursive function power() calculates by multiplying P with . The recursion stops when n becomes 0, where the result is 1.

c) A HR department stores employee details in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Input:

```
#T.Naga Siddhardha
def search_employee(ids, key, index):
    if index == len(ids):
        return False

    if ids[index] == key:
        return True

    return search_employee(ids, key, index + 1)


ids = list(map(int, input("Enter employee IDs:").split()))
key = int(input("Enter employee ID to search: "))

if search_employee(ids, key, 0):
    print("Employee ID found")
else:
    print("Employee ID not found")
```

Output:

```
Enter employee IDs: 1 2 3 4 5 6
Enter employee ID to search: 4
Employee ID found
```

Explanation:

The function checks each employee ID recursively. If the ID matches the given ID, it returns True. If the end of the list is reached without a match, it returns False.

d) In a logistic department, the no of possible ways to arrange n diff parcels are calculated using the factorial formula(n!). Write a recursive formula to find the factorial of a given number

Input:

```
#T.Naga Siddhardha
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)


n = int(input("Enter a number: "))

print("Factorial =", factorial(n))
```

Output:

```
Enter a number: 3
Factorial = 6
```

Explanation:

The recursive function factorial() calculates by multiplying n with the factorial of n-1. The recursion stops when n becomes 0 or 1.

e) Print first n terms in a fibanocci number series

Input:

```
#T.Naga Siddhardha
def fib(n):
    if n <= 1:
        return n
    else:
        return fib(n - 1) + fib(n - 2)

print(fib(5))
```

Output:

```
Enter number of terms: 3
Fibonacci Series:
0 1 1
```

Explanation:

The recursive function fibonacci() returns the sum of the previous two Fibonacci terms. The first two terms are 0 and 1. The program calls the function for each term from 0 to n-1.

WEEK-2

a) Write the implementation for the linear search algorithm

INPUT:

```
#T.Naga Siddhardha
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1


arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter element to search: "))

result = linear_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

OUTPUT:

```
Enter elements: 1 2 3  4 5 6 7 8
Enter element to search: 3
Element found at index 2
```

EXPLANATION:

Linear search checks each element **one by one** from the beginning of the list until the required element is found or the list ends.

b) Write the implementation for the binary search algorithm

INPUT:

```python
#T.Naga Siddhardha
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = list(map(int, input("Enter sorted elements: ").split()))
key = int(input("Enter element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

OUTPUT:

```
Enter sorted elements: 1 2 3 4 5 6
Enter element to search: 3
Element found at index 2
```

EXPLANATION:

Binary search works only on a **sorted list**. The program first sorts the input elements using sort(). It then repeatedly checks the middle element and eliminates half of the search range until the element is found or the range becomes empty.

WEEK-3

a) Write a python program to implement bubble sort where the input elements are taken from the user.

Input:

```
#T.Naga Siddhardha
arr = list(map(int, input("Enter elements: ").split()))

n = len(arr)

for i in range(n - 1):
    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted elements:", arr)
```

Output:

```
Enter elements: 4 3 56 76 3
Sorted elements: [3, 3, 4, 56, 76]
```

Explanation:

Bubble sort repeatedly compares **adjacent elements** and swaps them if they are in the wrong order. After each pass, the largest unsorted element moves to its correct position.

b) Write a python program to implement insertion sort where the input elements are taken from the user.

Input:

```
#T.Naga Siddhardha
arr = list(map(int, input("Enter elements: ").split()))

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print("Sorted elements:", arr)
```

Output:

```
Enter elements: 12 56 43 58 96 23
Sorted elements: [12, 23, 43, 56, 58, 96]
```

Explanation:

Insertion sort takes one element at a time and **inserts it into its correct position** among the already sorted elements.

C) Write a python program to implement selection sort where the input elements are taken from the user.

Input:

```
#T.Naga Siddhardha
arr = list(map(int, input("Enter elements: ").split()))
n = len(arr)

for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted elements:", arr)
```

Output:

```
Enter elements: 34 12 11 8 45 74 23 10
Sorted elements: [8, 10, 11, 12, 23, 34, 45, 74]
```

Explantion:

Selection sort finds the **smallest element** from the unsorted portion and swaps it with the first element of that portion. This process continues until the entire list is sorted.

Week – 4

4a)

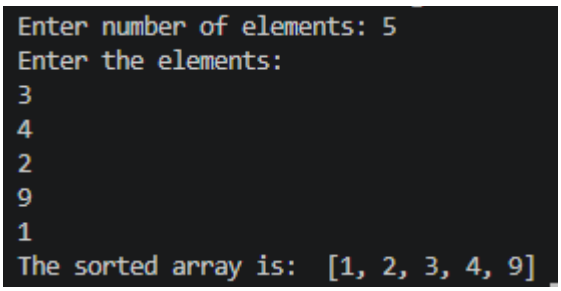
Aim:

Write a python program to implement merge sort where the input elements are taken from the user

Code:

```
#T.Naga Siddhardha
def MergeSort(array):
    if len(array)<=1:
        return array
    mid=len(array)//2
    left=array[:mid]
    right=array[mid:]
    MergeSort(left)
    MergeSort(right)
    i=j=k=0
    while i<len(left) and j<len(right):
        if left[i]<=right[j]:
            array[k]=left[i]
            i+=1
        else:
            array[k]=right[j]
            j+=1
        k+=1
    while i<len(left):
        array[k]=left[i]
        i+=1
        k+=1
    while j<len(right):
        array[k]=right[j]
        j+=1
        k+=1
    return array
array=[]
n=int(input("Enter number of elements: "))
print("Enter the elements: ")
for i in range(n):
    array.append(int(input()))
result=MergeSort(array)
print("The sorted array is: ",array)
```

Output:



Error Table:

Error	Rectification
RecursionError: maximum recursion depth exceeded	base case in merge_sort() len(arr) <1 to len(arr) <= 1
PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4> python merge_sort.py [2, 3, 7, 7, 8, 3, 7, 7, 8, 7, 7, 8]	kept the extend in the while loop instead of out of while loop

Description:

- The program implements the **Merge Sort** algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The merge_sort() function recursively divides the array into two halves.
- The array is divided until each sub-array contains one or zero elements.
- The left and right portions are then sorted recursively.
- The sorted left and right arrays are compared element by element.
- The smaller element is placed into the original array.
- Remaining elements from either portion are then copied into the array.
- Finally, the completely sorted array is displayed.
- Merge Sort follows the **divide-and-conquer** approach.

4b)

Aim:

Write a python program to implement quick sort where the input elements are taken from the user.

Code:

```
#T.Naga Siddhardha
Windsurf: Refactor | Explain | Generate Docstring | ×
def QuickSort(array,low,high):
    if low<high:
        i=low
        j=high
        pivot=low
        while i<j:
            while i<len(array) and array[i]<=array[pivot]:
                i+=1
            while array[j]>array[pivot]:
                j-=1
            if i<j:
                array[i],array[j]=array[j],array[i]
        array[j],array[pivot]=array[pivot],array[j]
        QuickSort(array,low,j-1)
        QuickSort(array,j+1,high)
array=list(map(int,input("Enter the elements (seperated with space): ").split(" ")))
length=len(array)
QuickSort(array,0,length-1)
print("Sorted array: ")
print(array)
```

Output:

```
Enter the elements (seperated with space): 5 3 8 4 9 6 1
Sorted array:
[1, 3, 4, 5, 6, 8, 9]
```

Error Table:

Error	Rectification
PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4> python quick_sort.py rolln0 : cse25142 name : P. Bhanu Kiran Enter size of array: 7 enter 1 element: 2 enter 2 element: 3	Instead of inputting the integers, inputted the string added int before the input arr.append(int(input(f"enter {i+1} element"))

enter 3 element: 56 enter 4 element: 9 enter 5 element: -` enter 6 element: enter 7 element: 2	
PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4> python quick_sort.py Traceback (most recent call last): File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 24, in <module> main() File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 5, in main quick_sort(arr) TypeError: quick_sort() missing 2 required positional arguments: 'low' and 'high' PS C:\Users\Lab\Documents\cse25142\dsa-lab\week-4>	added the two parameters to the function
Traceback (most recent call last): File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 29, in <module> main() File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 9, in main print(quick_sort(arr, 0, n-1)) ^^^^^^^^^^^^^^^^^^^^^^ File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 26, in quick_sort quick_sort(a, j+1, high) #right part ^^^^^^^^^^^^^^^^^^^^^^ File "C:\Users\Lab\Documents\cse25142\dsa-lab\week-4\quick_sort.py", line 24, in quick_sort	Placed a comma

a[j], a[pivot] = a[pivot] = a[j] # placing the pivot in the correct position ^^^^^^^^^^^^^^ ValueError: too many values to unpack (expected 2)	

Description:

- The program implements the **Quick Sort** algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The quick_sort() function is used to sort the array recursively.
- The first element of the current portion is selected as the **pivot**.
- Two pointers, i and j, are used to compare elements with the pivot.
- i moves from the left side and searches for an element greater than or equal to the pivot.
- j moves from the right side and searches for an element smaller than or equal to the pivot.
- If i < j, the two elements are swapped.
- After the partitioning process, the pivot is placed in its correct position.
- The array is then divided into left and right portions.
- Quick Sort is recursively applied to both portions.
- Finally, the sorted array is printed.

Week – 5

5a)

Aim:

Write a Python program to implement a Singly Linked List (SLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Code:

```python
#T.Naga Siddhardha
Windsurf: Refactor | Explain
class Node:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self,data):
        self.data=data
        self.next=None
Windsurf: Refactor | Explain
class SinglyLinkedList:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self):
        self.head=None
    Windsurf: Refactor | Explain | Generate Docstring | X
    def create(self):
        n=int(input("Enter number of elements: "))
        for i in range(n):
            x=int(input("Enter value: "))
            new=Node(x)
            if self.head is None:
                self.head=new
            else:
                temp=self.head
                while temp.next:
                    temp=temp.next
                temp.next=new
    Windsurf: Refactor | Explain | Generate Docstring | X
    def insertAtBeginning(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            new.next=self.head
            self.head=new
    Windsurf: Refactor | Explain | Generate Docstring | X
    def insertAtEnd(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
```

```
        else:
            temp=self.head
            while temp.next:
                temp=temp.next
            temp.next=new
    Windsurf: Refactor | Explain | Generate Docstring | X
    def insertAtIndex(self,index,data):
        new=Node(data)
        if index==0:
            self.insertAtBeginning(data)
            return
        elif index<0 or index>self.count():
            print("Invalid index")
            return
        else:
            temp=self.head
            for i in range(index-1):
                temp=temp.next
            new.next=temp.next
            temp.next=new
    Windsurf: Refactor | Explain | Generate Docstring | X
    def count(self):
        if self.head is None:
            return 0
        else:
            n=0
            temp=self.head
            while temp:
                n+=1
                temp=temp.next
            return n
    Windsurf: Refactor | Explain | Generate Docstring | X
    def deleteAtBeginning(self):
        if self.head is None:
            print("No data to delete")
```

```
        else:
            temp=self.head
            self.head=temp.next
            print("Deleted value = ",temp.data)
    Windsurf: Refactor | Explain | Generate Docstring | X
    def deleteAtEnd(self):
        if self.head is None:
            print("No data to delete")
        elif self.head.next is None:
            print("Deleted value = ",self.head.data)
            self.head=None
        else:
            temp=self.head
            temp1=temp
            while temp.next:
                temp1=temp
                temp=temp.next
            print("Deleted value = ",temp.data)
            temp1.next=None
    Windsurf: Refactor | Explain | Generate Docstring | X
    def delete(self,value):
        if self.head is None:
            print("No data to delete")
        else:
            temp=self.head
            if temp.data==value:
                temp.next=self.head
                print("Value deleted")
                return
            while temp.next and temp.next.data!=value:
                temp=temp.next
            if temp.next is None:
                print("Value not present")
```

```
            else:
                temp.next=temp.next.next
                print("Value deleted")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def display(self):
        if self.head is None:
            print("No data")
        else:
            temp=self.head
            while temp:
                print(temp.data,end="->")
                temp=temp.next
            print("None")
SLL=SinglyLinkedList()
while True:
    print("1. Create linked list")
    print("2. Insert beginning")
    print("3. Insert end")
    print("4. Insert at index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display/Traverse")
    print("10. Exit")
    choice=int(input("Enter your choice:"))
    if choice==1:
        SLL.create()
    elif choice==2:
        val=int(input("Enter the value:"))
        SLL.insertAtBeginning(val)
    elif choice==3:
        val=int(input("Enter the value:"))
        SLL.insertAtEnd(val)
```

```
    elif choice==4:
        ind=int(input("Enter the index:"))
        val=int(input("Enter the value:"))
        SLL.insertAtIndex(ind,val)
    elif choice==5:
        val=int(input("Enter value to be deleted:"))
        SLL.delete(val)
    elif choice==6:
        SLL.deleteAtBeginning()
    elif choice==7:
        SLL.deleteAtEnd()
    elif choice==8:
        print(f"Number of nodes = {SLL.count()}")
    elif choice==9:
        SLL.display()
    elif choice==10:
        break
    else:
        print("Invalid choice")
```

```
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit
Enter your choice:1
Enter number of elements: 10
Enter value: 1
Enter value: 2
Enter value: 3
Enter value: 4
Enter value: 5
Enter value: 6
Enter value: 7
Enter value: 8
Enter value: 9
Enter value: 10
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit
Enter your choice:2
Enter the value:0
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
```

```
8. Count number of nodes
9. Display/Traverse
10. Exit
Enter your choice:3
Enter the value:11
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit
Enter your choice:4
Enter the index:4
Enter the value:5
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit
Enter your choice:5
Enter value to be deleted:10
Value deleted
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
```

```
10. Exit
Enter your choice:6
Deleted value =  0
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit
Enter your choice:7
Deleted value =  11
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit
Enter your choice:8
Number of nodes = 10
1. Create linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit
Enter your choice:9
1->2->3->5->4->5->6->7->8->9->None
1. Create linked list
```

Error Table:

Error	Rectification
The program may display an incorrect message for insertion at a specific index even when the index is invalid	Check whether the index is valid before displaying the successful insertion message.

Description:

- The program implements a Singly Linked List using Python classes.
- The Node class is used to create individual nodes.

- Each node contains two parts: data and next.
- data stores the value of the node.
- next stores the reference to the next node.
- The SingleLinkedList class manages the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a linked list.
- It supports insertion at the beginning of the list.
- It supports insertion at the end of the list.
- It supports insertion at a specific index.
- It supports deletion of a specific value.
- It supports deletion of the first node.
- It supports deletion of the last node.
- It provides a function to count the number of nodes.
- It provides a function to display/traverse all the nodes.
- A menu-driven approach is used to allow the user to select different operations.
- The program continues running until the user selects the Exit option

5b)

Aim:

Write a Python program to implement a Double Linked List (DLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Code:

```python
#T.Naga Siddhardha
Windsurf: Refactor | Explain
class Node:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,data):
        self.prev=None
        self.data=data
        self.next=None
Windsurf: Refactor | Explain
class DoublyLinkedList:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self):
        self.head=None
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def insert_begin(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            new.next=self.head
            self.head.prev=new
            self.head=new
        print("Insertion completed")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def insert_end(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            temp=self.head
            while temp.next:
                temp=temp.next
            temp.next=new
            new.prev=temp
        print("Insertion completed")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def insert_index(self,index,data):
        if index==0:
            self.insert_begin(data)
```

```
            return
        elif index<0 or index>self.count():
            print("Invalid index")
            return
        else:
            new=Node(data)
            temp=self.head
            for i in range(index-1):
                temp=temp.next
            new.next=temp.next
            new.prev=temp
            if temp.next:
                temp.next.prev=new
            temp.next=new
        print("Insertion completed")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def delete_beginning(self):
        if self.head is None:
            print("No elements to delete")
            return
        elif self.head.next is None:
            self.head=None
        else:
            self.head.next.prev=None
            self.head=self.head.next
        print("Value deleted")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def delete_end(self):
        if self.head is None:
            print("No elements to delete")
            return
        elif self.head.next is None:
            self.head=None
        else:
            temp=self.head
            while temp.next:
```

```
            temp=temp.next
        temp.prev.next=None
        temp.prev=None
    print("Value deleted")
Windsurf: Refactor | Explain | Generate Docstring | ×
def delete_index(self,index):
    if index==0:
        self.delete_beginning()
        return
    elif index<0 or index>=self.count():
        print("Invalid index")
        return
    else:
        temp=self.head
        for i in range(index-1):
            temp=temp.next
        delete_node=temp.next
        temp.next=delete_node.next
        if delete_node.next:
            delete_node.next.prev=temp
        delete_node.prev=None
        delete_node.next=None
    print("Value deleted")
Windsurf: Refactor | Explain | Generate Docstring | ×
def count(self):
    if self.head is None:
        return 0
    else:
        n=0
        temp=self.head
        while temp:
            n+=1
            temp=temp.next
        return n
Windsurf: Refactor | Explain | Generate Docstring | ×
def display_forward(self):
```

```
        if self.head is None:
            print("No elements in Doubly linked list")
        else:
            temp=self.head
            print("None",end="<->")
            while temp:
                print(temp.data,end="<->")
                temp=temp.next
            print("None")
    Windsurf: Refactor | Explain | Generate Docstring | X
    def display_backward(self):
        if self.head is None:
            print("No elements in doubly linked list")
        else:
            temp=self.head
            while temp.next:
                temp=temp.next
            print("None",end="<->")
            while temp:
                print(temp.data,end="<->")
                temp=temp.prev
            print("None")
DLL=DoublyLinkedList()
while True:
    print("1. Insert at beginning")
    print("2. Insert at ending")
    print("3. Insert at index")
    print("4. Delete at beginning")
    print("5. Delete at ending")
    print("6. Delete at index")
    print("7. Count")
    print("8. Display forward")
    print("9. Display backward")
    print("10. Exit")
    choice=int(input("Enter your choice"))
    if choice==1:
```

```
        value=int(input("Enter a value: "))
        DLL.insert_begin(value)
    elif choice==2:
        value=int(input("Enter a value: "))
        DLL.insert_end(value)
    elif choice==3:
        index=int(input("Enter index: "))
        value=int(input("Enter a value: "))
        DLL.insert_index(index,value)
    elif choice==4:
        DLL.delete_beginning()
    elif choice==5:
        DLL.delete_end()
    elif choice==6:
        index = int(input("Enter index: "))
        DLL.delete_index(index)
    elif choice==7:
        print(f"The number of elements: {DLL.count()}")
    elif choice==8:
        DLL.display_forward()
    elif choice==9:
        DLL.display_backward()
    elif choice==10:
        break
    else:
        print("Invalid choice")
```

Output:

```
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice1
Enter a value: 6
Insertion completed
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice1
Enter a value: 4
Insertion completed
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice1
Enter a value: 3
Insertion completed
1. Insert at beginning
2. Insert at ending
```

```
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice1
Enter a value: 2
Insertion completed
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice1
Enter a value: 1
Insertion completed
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice2
Enter a value: 7
Insertion completed
1. Insert at beginning
```

```
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice3
Enter index: 4
Enter a value: 5
Insertion completed
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice4
Value deleted
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice5
Value deleted
1. Insert at beginning
```

```
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice6
Enter index: 7
Invalid index
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice8
None<->2<->3<->4<->5<->6<->None
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Delete at index
7. Count
8. Display forward
9. Display backward
10. Exit
Enter your choice9
None<->6<->5<->4<->3<->2<->None
1. Insert at beginning
2. Insert at ending
3. Insert at index
```

Error Table:

Error	Rectification
The prev and next links may not be updated correctly during insertion or deletion.	Update both links whenever a node is inserted or deleted.

Description:

- The program implements a Doubly Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains three parts: data, prev, and next.

- data stores the value of the node.
- prev stores the reference to the previous node.
- next stores the reference to the next node.
- The DoubleLinkedList class is used to manage the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a doubly linked list.
- It supports insertion at the beginning of the list.
- It supports insertion at the end of the list.
- It supports insertion at a specific index.
- It supports deletion of a specific value.
- It supports deletion of the first node.
- It supports deletion of the last node.
- It provides a function to count the number of nodes.
- It provides a function to display/traverse the list.
- During insertion and deletion, both prev and next links are updated.
- The list is displayed using <--> to show the connection between nodes.
- A menu-driven approach is used to perform different DLL operations.
- The program continues until the user selects the Exit option.

5c)

Aim:

Write a Python program to implement a Circular Linked List (CLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse

10. Exit

Code:

```
#T.Naga Siddhardha
Windsurf: Refactor | Explain
class Node:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,data):
        self.data=data
        self.next=None
Windsurf: Refactor | Explain
class CircularSingleLinkedList:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self):
        self.head=None
        self.tail=None
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def insert_begin(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
            self.tail=new
            new.next=self.head
        else:
            new.next=self.head
            self.head=new
            self.tail.next=self.head
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def insert_end(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
            self.tail=new
            new.next=self.head
        else:
            new.next=self.head
            self.tail.next=new
            self.tail=new
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def insert_index(self,index,data):
        new=Node(data)
        if index==0:
```

```
            self.insert_begin(data)
            return
        elif index<0 or index>self.count():
            print("Invalid index")
            return
        else:
            temp=self.head
            for i in range(index-1):
                temp=temp.next
                if temp==self.head:
                    print("Invalid index")
                    return
            new.next=temp.next
            temp.next=new
            if temp==self.tail:
                self.tail=new
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def delete_begin(self):
        if self.head is None:
            print("No elements to delete")
            return
        elif self.head==self.tail:
            self.head=None
            self.tail=None
        else:
            self.head=self.head.next
            self.tail.next=self.head
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def delete_end(self):
        if self.head is None:
            print("No elements to delete")
            return
        elif self.head==self.tail:
            self.head=None
            self.tail=None
```

```
        else:
            temp=self.head
            while temp.next!=self.tail:
                temp=temp.next
            temp.next=self.head
            self.tail=temp
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def count(self):
        if self.head is None:
            return 0
        else:
            n=0
            temp=self.head
            while True:
                n+=1
                temp=temp.next
                if temp==self.head:
                    break
            return n
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def traverse(self):
        if self.head is None:
            print("No elements")
            return
        else:
            temp=self.head
            while True:
                print(temp.data,end="->")
                temp=temp.next
                if temp==self.head:
                    break
            print("back to end")
```

```
    def displayHeadTail(self):
        if self.head is None:
            print("No elements")
            return
        else:
            print("Head: ",self.head.data)
            print("Tail: ",self.tail.data)
            print("Tail.next: ",self.tail.next.data)
CSLL=CircularSingleLinkedList()
while True:
    print("1. Insert at beginning")
    print("2. Insert at ending")
    print("3. Insert at index")
    print("4. Delete at beginning")
    print("5. Delete at ending")
    print("6. Count")
    print("7. Display all elements")
    print("8. Display head and tail")
    print("9. Exit")
    choice=int(input("Enter your choice"))
    if choice==1:
        data=int(input("Enter value: "))
        CSLL.insert_begin(data)
    elif choice==2:
        data=int(input("Enter value: "))
        CSLL.insert_end(data)
    elif choice==3:
        index=int(input("Enter index: "))
        data=int(input("Enter value: "))
        CSLL.insert_index(index,data)
    elif choice==4:
        CSLL.delete_begin()
    elif choice==5:
        CSLL.delete_end()
    elif choice==6:
        print(f"Number of elements: {CSLL.count()}")
```

```
elif choice==7:
    CSLL.traverse()
elif choice==8:
    CSLL.displayHeadTail()
elif choice==9:
    break
else:
    print("Invalid choice")
```

Output:

```
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice1
Enter value: 6
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice1
Enter value: 4
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice1
Enter value: 3
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
```

```
Enter your choice1
Enter value: 2
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice1
Enter value: 1
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice2
Enter value: 7
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice3
Enter index: 4
Enter value: 5
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
```

```
Enter your choice4
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice5
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice6
Number of elements: 5
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
7. Display all elements
8. Display head and tail
9. Exit
Enter your choice7
2->3->4->5->6->back to end
1. Insert at beginning
2. Insert at ending
3. Insert at index
4. Delete at beginning
5. Delete at ending
6. Count
```

Error Table:

Error	Rectification
No errors	

Description:

- The program implements a Circular Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains two parts: data and next.
- data stores the value of the node.
- next stores the reference to the next node.
- In a Circular Linked List, the last node points back to the first node.

- The CircularLinkedList class is used to manage the linked list.
- The head variable stores the reference to the first node.
- The program provides an option to create a circular linked list.
- It supports insertion at the beginning of the list.
- It supports insertion at the end of the list.
- It supports insertion at a specific index.
- It supports deletion of a specific value.
- It supports deletion of the first node.
- It supports deletion of the last node.
- It provides a function to count the number of nodes.
- It provides a function to display/traverse the circular linked list.
- During traversal, the program stops when it reaches the head again.
- The list is displayed using --> and ends with HEAD, showing its circular nature.
- The program handles the special case where the list contains only one node.
- A menu-driven approach is used to perform different CLL operations.
- The program continues until the user selects the Exit option.

Week – 6

6a)

Aim:

Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array.

Code:

```
#T.Naga Siddhardha
Windsurf: Refactor | Explain
class StackArray:
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def __init__(self,size):
        self.size=size
        self.stack=[None]*size
        self.top=-1
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def push(self,data):
        if self.top==self.size-1:
            print("Stack overflow")
        else:
            self.top+=1
            self.stack[self.top]=data
            print(f"{data} is pushed into the stack")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def pop(self):
        if self.top==-1:
            print("Stack underflow")
        else:
            item=self.stack[self.top]
            self.stack[self.top]=None
            self.top-=1
            print(f"{item} is popped from the stack")
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def peek(self):
        if self.top==-1:
            print("No elements in the stack")
        else:
            print("Top element: ",self.stack[self.top])
    Windsurf: Refactor | Explain | Generate Docstring | ×
    def display(self):
        if self.top==-1:
            print("No elements in the stack")
        else:
            print("Elements in the stack are: ")
            for i in range(self.top,-1,-1):
                print(self.stack[i])
```

```
size=int(input("Enter the size of the stack: "))
stack=StackArray(size)
while True:
    print("-"*10,end='')
    print("Stack Menu",end='')
    print("-"*10)
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice=int(input("Enter your choice: "))
    if choice==1:
        item=int(input("Enter the element: "))
        stack.push(item)
    elif choice==2:
        stack.pop()
    elif choice==3:
        stack.peek()
    elif choice==4:
        stack.display()
    elif choice==5:
        break
    else:
        print("Invalid choice")
```

Output:

```
Enter the size of the stack: 5
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 1
1 is pushed into the stack
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 2
2 is pushed into the stack
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 3
3 is pushed into the stack
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 4
4 is pushed into the stack
----------Stack Menu----------
1. Push
2. Pop
3. Peek
```

```
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 5
5 is pushed into the stack
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
5 is popped from the stack
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element:  4
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Elements in the stack are:
4
3
2
1
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
```

Error Table:

Error	Rectification
In peek, self.peek is taken in ‘if’ does not give correct answer for stack underflow.	Self.top is taken in ‘if’ to give correct output.

Description:

- The Stack class implements a stack using a Python list.
- size stores the maximum size of the stack.
- stack stores the stack elements.
- top keeps track of the top position. Initially, top = -1, meaning the stack is empty.

- **Push** adds an element to the top of the stack.
- **Pop** removes the top element from the stack.
- **Peek** displays the top element without removing it.
- **Display** prints all elements from top to bottom.
- **Stack Overflow** occurs when trying to push into a full stack.
- **Stack Underflow** occurs when trying to pop from an empty stack.
- The program uses a menu-driven approach with options for Push, Pop, Peek, Display, and Exit.

6b)

Aim:

Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Linked List

Code:

```python
#T.Naga Siddhardha
Windsurf: Refactor | Explain
class Node:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self,data):
        self.data=data
        self.next=None
Windsurf: Refactor | Explain
class StackLinkedList:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self):
        self.top=None
    Windsurf: Refactor | Explain | Generate Docstring | X
    def push(self,data):
        new=Node(data)
        new.next=self.top
        self.top=new
        print(f"{data} pushed into the stack")
    Windsurf: Refactor | Explain | Generate Docstring | X
    def pop(self):
        if self.top is None:
            print("Stack underflow")
        else:
            temp=self.top
            print(f"{temp.data} is popped from the stack")
            self.top=self.top.next
    Windsurf: Refactor | Explain | Generate Docstring | X
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print(f"Top element: {self.top.data}")
    Windsurf: Refactor | Explain | Generate Docstring | X
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp=self.top
            print("Stack elements are: ")
```

```
        if self.top is None:
            print("Stack is empty")
        else:
            temp=self.top
            print("Stack elements are: ")
            while temp is not None:
                print(temp.data)
                temp=temp.next
stack=StackLinkedList()
while True:
    print("-"*10,end='')
    print("Stack Menu",end='')
    print("-"*10)
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice=int(input("Enter your choice: "))
    if choice==1:
        item=int(input("Enter a value: "))
        stack.push(item)
    elif choice==2:
        stack.pop()
    elif choice==3:
        stack.peek()
    elif choice==4:
        stack.display()
    elif choice==5:
        break
    else:
        print("Invalid choice")
```

Output:

```
Enter your choice: 1
Enter a value: 5
5 pushed into the stack
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
5 is popped from the stack
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 4
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
4
3
2
1
----------Stack Menu----------
1. Push
2. Pop
3. Peek
4. Display
5. Exit
```

Error Table:

Error	Rectification
No errors	

Description:

- The stack is implemented using a **singly linked list**.
- Each node contains data and a next pointer.
- top points to the top node of the stack.
- **Push** inserts a new node at the beginning of the linked list.
- **Pop** removes the node pointed to by top.
- **Peek** displays the data of the top node without removing it.
- **Display** traverses the linked list from top to the last node.

- If top is None, the stack is empty.
- Stack Overflow is not a fixed-size issue here because the stack grows dynamically, subject to available memory.

WEEK-7

a. Write a Python program to implement a Queue and its operations such as Enqueue, Dequeue, Peek, and Display using a menu-driven approach. Implement the Queue using both an Array and a Linked List data structure.

Input:

```
#T.Naga Siddhardha
class QueueArray:
    def __init__(self,size):
        self.size=size
        self.queue=[None]*size
        self.front=-1
        self.rear=-1
    def enqueue(self,data):
        if self.rear==self.size-1:
            print("Queue overflow")
        else:
            if self.front==-1:
                self.front=0
            self.rear+=1
            self.queue[self.rear]=data
            print(f"{data} is inserted into the queue")
    def dequeue(self):
        if self.front==-1 or self.front>self.rear:
            print("Queue underflow")
        else:
            element=self.queue[self.front]
            self.queue[self.front]=None
            self.front+=1
            print(f"{element} deleted from the queue")
            if self.front>self.rear:
                self.front=-1
                self.rear=-1
    def peek(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            print("Front element: ",self.queue[self.front])
    def display(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            print("The elements of the queue are: ")
            for i in range(self.front,self.rear+1,1):
                print(self.queue[i])
size=int(input("Enter the size of the queue: "))
queue=QueueArray(size)
while True:
    print('-'*10,end='')
    print("Queue menu",end='')
    print('-'*10)
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
```

```
    print("4. Display")
    print("5. Exit")
    choice=int(input("Enter your choice: "))
    if choice==1:
        data=int(input("Enter the value: "))
        queue.enqueue(data)
    elif choice==2:
        queue.dequeue()
    elif choice==3:
        queue.peek()
    elif choice==4:
        queue.display()
    elif choice==5:
        break
    else:
        print("Invalid choice")
```

Output:

```
--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 1
Element inserted successfully

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted successfully

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 1

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
```

```
Enter element: 3
Element inserted successfully

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 2

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
2 3

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

Explanation:

This program implements a **FIFO (First In First Out) queue using an array/list** in Python with a menu-driven approach. The **enqueue operation** inserts a new element at the rear of the queue, while the **dequeue operation** removes an element from the front. The **peek operation** displays the front element without removing it from the queue. The **display operation** prints all the elements currently present from front to rear. The program also checks for **queue overflow and underflow** conditions before performing operations. The menu repeatedly accepts the user's choice and performs the selected operation until the user chooses Exit.

Errors Faced

While implementing the array queue, errors such as **Queue Overflow** can occur when the rear reaches the maximum queue size, and **Queue Underflow** can occur when trying to dequeue from an empty queue. Incorrect handling of the front and rear variables can also cause indexing problems or incorrect queue output. Another possible error is **ValueError**, which occurs when the user enters a non-numeric value where an integer is expected. Invalid menu choices can also cause incorrect operation selection, so an appropriate validation message is displayed.

LINKED LIST:

Input:

```
#T.Naga Siddhardha
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None
class QueueSingleLinkedList:
    def __init__(self):
        self.front=None
        self.rear=None
    def enqueue(self,data):
        new=Node(data)
        if self.rear is None:
            self.front=self.rear=new
        else:
            self.rear.next=new
            self.rear=new
        print(f"{data} is inserted into the queue.")
    def dequeue(self):
        if self.front is None:
            print("Queue is empty")
        else:
            element=self.front.data
            self.front=self.front.next
            print(f"{element} is deleted from the queue.")
            if self.front is None:
                self.rear=None
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element: ",self.front.data)
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements in the queue are: ")
            temp=self.front
            while temp is not None:
                print(temp.data)
                temp=temp.next
queue=QueueSingleLinkedList()
while True:
    print('-'*10,end='')
    print("Queue Menu",end='')
    print('-'*10)
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
```

```
    print("4. Display")
    print("5. Exit")
    choice=int(input("Enter your choice: "))
    if choice==1:
        data=int(input("Enter the value: "))
        queue.enqueue(data)
    elif choice==2:
        queue.dequeue()
    elif choice==3:
        queue.peek()
    elif choice==4:
        queue.display()
    elif choice==5:
        break
    else:
        print("Invalid choice")
```

Output:

```
--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted successfully

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 2
Element inserted successfully

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 2
```

```
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 4
Element inserted successfully

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 2

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements:
2 4

--- QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

Explanation

This program implements a **FIFO queue using a linked list** where every element is stored inside a node. Each node contains two parts: **data** and a reference to the next node. The front pointer points to the first node and the rear pointer points to the last node. During **enqueue**, a new node is created and

attached at the rear, while during **dequeue**, the front node is removed and the front pointer is moved to the next node. The **peek** operation displays the data of the front node without deleting it, and **display** traverses the linked list from front to rear. Since memory is allocated dynamically, the linked-list queue does not require a fixed queue size.

Errors Faced

While implementing the linked-list queue, the main errors faced were related to **front and rear pointer management**. Queue Underflow can occur when dequeue or peek is performed while the queue is empty. Special care is required when deleting the last element because both front and rear must be changed to None. Incorrectly accessing a node or its next reference may result in an **AttributeError**. A **ValueError** may also occur if non-integer data is entered when the program expects an integer. Incorrect pointer updates can also cause elements to be skipped or the linked list to display incorrectly.

b.Write a Python program to implement a Circular Queue and its operations such as Enqueue, Dequeue, Peek, and Display using a menu-driven approach.

Input:

```
#T.Naga Siddhardha
class CircularQueue:
    def __init__(self,size):
        self.size=size
        self.queue=[None]*size
        self.front=-1
        self.rear=-1
    def enqueue(self,data):
        if (self.rear+1)%self.size==self.front:
            print("Queue overflow")
        elif self.front==-1:
            self.front=0
            self.rear=0
            self.queue[self.rear]=data
            print(data," inserted into the queue.")
        else:
            self.rear=(self.rear+1)%self.size
            self.queue[self.rear]=data
            print(data," inserted into the queue.")
    def dequeue(self):
        if self.front==-1:
            print("Queue underflow")
        elif self.front==self.rear:
            element=self.queue[self.front]
            self.queue[self.front]=None
            self.front=-1
            self.rear=-1
            print(f"{element} deleted from the queue")
        else:
            element=self.queue[self.front]
            self.queue[self.front]=None
            self.front=(self.front+1)%self.size
            print(f"{element} deleted from the queue")
    def display(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            print("Elements in the queue are: ")
            i=self.front
            while True:
                print(self.queue[i])
                if i==self.rear:
                    break
                i=(i+1)%self.size
size=int(input("Enter the size of the queue: "))
queue=CircularQueue(size)
while True:
    print('-'*10,end='')
    print("Queue menu",end='')
```

```
    print("Queue menu",end='')
    print('-'*10)
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice=int(input("Enter your choice: "))
    if choice==1:
        data=int(input("Enter the value: "))
        queue.enqueue(data)
    elif choice==2:
        queue.dequeue()
    elif choice==3:
        queue.display()
    elif choice==4:
        break
    else:
        print("Invalid choice")
```

Output:

```
--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 3
Element inserted successfully

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 4
Element inserted successfully

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 5
Element inserted successfully
```

```
--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 3

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 4

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements:
4 5

--- CIRCULAR QUEUE ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

Explanation:

This program implements a **circular queue using an array** and follows the FIFO principle. Unlike a normal linear queue, a circular queue allows the rear to move back to the beginning of the array when it reaches the last position. The **enqueue operation** inserts elements at the rear using the modulo operation, while **dequeue** removes elements from the front and moves the front circularly. The **peek operation** displays the current front element without deleting it, and **display** traverses the queue from front to rear while handling the circular arrangement. The condition (rear + 1) % MAX == front is used to identify a full circular queue, while front == -1 represents an empty queue.

Errors Faced :

While implementing the circular queue, errors can occur if the **full and empty conditions** are not handled correctly. Circular Queue Overflow occurs when the next position of the rear is equal to the front, while Underflow occurs when the queue is empty. Incorrect use of the modulo operator can cause the front or rear to move to the wrong position. The display operation may also produce an infinite loop if its stopping condition is incorrect, especially when the rear wraps around to the beginning of the array. A **ValueError** can occur when the user enters invalid non-numeric input, and invalid menu choices must also be handled properly

Week – 8

8a)

Aim:

Write a python program to implement a binary tree with its operations using an array.

Code:

```
#T.Naga Siddhardha
class BinaryTree:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    # Insert a node
    def insert(self, value):
        for i in range(self.size):
            if self.tree[i] is None:
                self.tree[i] = value
                return
        print("Tree is full")

    # Search a node
    def search(self, value):
        for i in range(self.size):
            if self.tree[i] == value:
                return i
        return -1

    # Inorder Traversal
    def inorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        self.inorder(2 * i + 1)
        print(self.tree[i], end=" ")
        self.inorder(2 * i + 2)

    # Preorder Traversal
    def preorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        print(self.tree[i], end=" ")
        self.preorder(2 * i + 1)
        self.preorder(2 * i + 2)

    # Postorder Traversal
    def postorder(self, i=0):
        if i >= self.size or self.tree[i] is None:
            return

        self.postorder(2 * i + 1)
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")

    # Display array
```

```
    # Display array
    def display(self):
        print(self.tree)


# Main program
bt = BinaryTree(10)

bt.insert(25)
bt.insert(15)
bt.insert(40)
bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(50)

print("Array representation:")
bt.display()

print("\nInorder:")
bt.inorder()

print("\nPreorder:")
bt.preorder()

print("\nPostorder:")
bt.postorder()

print("\n\nSearching for 30:")

position = bt.search(30)

if position != -1:
    print("30 found at index", position)
else:
    print("30 not found")
```

Output:

```
Array representation:
[25, 15, 40, 10, 20, 30, 50, None, None, None]

Inorder:
10 15 20 25 30 40 50
Preorder:
25 15 10 20 40 30 50
Postorder:
10 20 15 30 50 40 25

Searching for 30:
30 found at index 5
```

Error Table:

Error	Rectification
TypeError: BinaryTree.level_order() takes 1 positional argument but 2 were given	remove the argument in level order function call

level order: A, C, F, E, B, D, in level order, instead of removing first node removed the last node	given 0 as a argument in pop to remove the first node

Description:

- Root Node: The root of the tree is always stored at index `0`.
- Left Child: For any node located at index `i`, its left child is placed at index `2i + 1`.
- Right Child: For any node located at index `i`, its right child is placed at index `2i + 2`.
- Parent Node: For any child node at index `i` (where `i > 0`), its parent is found at index `(i - 1) / 2` (using integer division).
- Empty Nodes: If a node doesn't exist, its corresponding array index is left empty (usually represented by `null` or a sentinel value).

8b)

Aim:

Write a python program to implement a binary tree with its operations using a linked list

Code:

```
#T.Naga Siddhardha
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    # Insert a node
    def insert(self, value):
        new_node = Node(value)

        if self.root is None:
            self.root = new_node
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
                current.left = new_node
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                return
            else:
                queue.append(current.right)

    # Search a node
    def search(self, value):
        if self.root is None:
            return False

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.data == value:
                return True
```

```
            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        return False

    # Inorder Traversal
    def inorder(self, node):
        if node is not None:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    # Preorder Traversal
    def preorder(self, node):
        if node is not None:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    # Postorder Traversal
    def postorder(self, node):
        if node is not None:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    # Level Order Traversal
    def level_order(self):
        if self.root is None:
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            print(current.data, end=" ")

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)


# Main program
```

```
bt = BinaryTree()

bt.insert(25)
bt.insert(15)
bt.insert(40)
bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(50)

print("Inorder:")
bt.inorder(bt.root)

print("\nPreorder:")
bt.preorder(bt.root)

print("\nPostorder:")
bt.postorder(bt.root)

print("\nLevel Order:")
bt.level_order()

print("\n\nSearching for 30:")

if bt.search(30):
    print("30 found")
else:
    print("30 not found")
```

Output:

```
Inorder:
10 15 20 25 30 40 50
Preorder:
25 15 10 20 40 30 50
Postorder:
10 20 15 30 50 40 25
Level Order:
25 15 40 10 20 30 50

Searching for 30:
30 found
```

Error Table:

Error	Rectification
in order: D, B, A, F, C, E,	set left and set right functions exchanged for the index 2

Description:

- Node Structure: Each element in the tree is a "Node" object that stores three pieces of information: the actual data (value), a pointer/reference to the left child, and a pointer/reference to the right child.
- Root Pointer: The entire tree is accessed and traversed starting from a single main pointer called the root, which points to the very first node.

- Parent-Child Connections: Instead of doing math to find children like in an array, you simply follow the node's left or right pointer to memory addresses where the children are stored.
- Empty Nodes (Leaves): If a node doesn't have a left or right child, that specific pointer is simply set to null (or None/nil). A node where both pointers are null is a leaf.
- Best Use Case: This is the standard, most versatile way to build a binary tree. It is highly efficient for dynamic, unbalanced, or sparse trees because memory is only allocated exactly when a new node is created—meaning absolutely no memory is wasted on empty slots.

Week – 9

Write a python program to implement a binary search tree with its operations and traversals

Code:

```
#T.Naga Siddhardha
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
```

```
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
```

```
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)


def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)


# ======================================================
# MAIN PROGRAM
# ======================================================

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
```

```
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
    print("10. COUNT LEAVES")
    print("11. COUNT INTERNAL NODES")
    print("12. HEIGHT")
    print("13. LEVEL ORDER TRAVERSAL")
    print("14. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        print("Preorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            preorder(root)
            print()
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            inorder(root)
            print()
    elif choice == 4:
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
```

```
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output:

```
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 1
Element inserted successfully.


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 14
Program terminated.
```

Description:

- **Node Structure:** Each node stores a value (key) along with references to two children: left and right.
- **BST Property:** For any node, all keys in its left subtree are strictly smaller than its key, and all keys in its right subtree are strictly greater.
- **Core Operations:**
  - **Insertion:** Compares the new value with the current node; moves left if smaller, right if larger, and attaches at an empty spot (None).
  - **Search:** Follows the same direction rules, achieving average-case O(\log n) time complexity and worst-case O(n) if unbalanced.
- **In-Order Traversal (Left -> Root -> Right):**
  - Recursively visits the left subtree, processes the current node, then visits the right subtree.
  - **Key Feature:** Produces node values in sorted, ascending order.
- **Pre-Order Traversal (Root -> Left -> Right):**
  - Processes the current node first, then traverses left and right subtrees.
  - **Key Feature:** Useful for creating a copy or clone of the tree structure.
- **Post-Order Traversal (Left -> Right -> Root):**

- Traverses left and right subtrees before processing the current node.
- **Key Feature:** Commonly used for deleting/freeing tree nodes or evaluating expression trees bottom-up.

Week-10

AIM: Write a python program to implement a AVL tree with all its operations.

PROGRAM:

```
#T.Naga Siddhardha
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:

    # Get height of node
    def height(self, node):
        if node is None:
            return 0
        return node.height

    # Get balance factor
    def get_balance(self, node):
        if node is None:
            return 0
        return self.height(node.left) - self.height(node.right)

    # Right rotation
    def right_rotate(self, y):
        x = y.left
        t2 = x.right

        x.right = y
        y.left = t2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))

        return x

    # Left rotation
    def left_rotate(self, x):
        y = x.right
        t2 = y.left

        y.left = x
        x.right = t2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
```

```
        return y

    # Insert operation
    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)

        # LR Case
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RL Case
        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Find minimum value node
    def min_value_node(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current

    # Delete operation
    def delete(self, root, key):
```

```
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)

        elif key > root.key:
            root.right = self.delete(root.right, key)

        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.min_value_node(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        # LR Case
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Search operation
    def search(self, root, key):
        if root is None or root.key == key:
```

```
        if key < root.key:
            return self.search(root.left, key)

        return self.search(root.right, key)

    # Inorder traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    # Preorder traversal
    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Postorder traversal
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")


# Main program
tree = AVLTree()
root = None

while True:
    print("\n\n--- AVL TREE OPERATIONS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter value to insert: "))
        root = tree.insert(root, key)
        print("Inserted successfully.")
```

```
print("\n\n--- AVL TREE OPERATIONS ---")
print("1. Insert")
print("2. Delete")
print("3. Search")
print("4. Inorder Traversal")
print("5. Preorder Traversal")
print("6. Postorder Traversal")
print("7. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    key = int(input("Enter value to insert: "))
    root = tree.insert(root, key)
    print("Inserted successfully.")

elif choice == 2:
    key = int(input("Enter value to delete: "))
    root = tree.delete(root, key)
    print("Deletion operation completed.")

elif choice == 3:
    key = int(input("Enter value to search: "))
    result = tree.search(root, key)

    if result:
        print("Element found.")
    else:
        print("Element not found.")

elif choice == 4:
    print("Inorder Traversal:")
    tree.inorder(root)

elif choice == 5:
    print("Preorder Traversal:")
    tree.preorder(root)

elif choice == 6:
    print("Postorder Traversal:")
    tree.postorder(root)

elif choice == 7:
    print("Exiting...")
    break

else:
    print("Invalid choice.")
```

OUTPUT:

```
--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 1
Inserted successfully.


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 2
Inserted successfully.


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 2
Enter value to delete: 2
Deletion operation completed.


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 3
```

EXPLANATION:

- The Node class creates a node containing a key, left link, right link, and height.
- The height() function returns the height of a node and returns 0 if the node is empty.
- The get_balance() function calculates the balance factor using the heights of the left and right subtrees.
- The right_rotate() and left_rotate() functions perform rotations to balance the AVL tree.
- The insert() function inserts values into the AVL tree, placing smaller values on the left and larger values on the right.
- The LL, RR, LR, and RL cases handle tree imbalance using appropriate rotations.
- The min_value_node() function finds the minimum element by moving to the leftmost node.

- The max_value_node() function finds the maximum element by moving to the rightmost node.
- The delete() function removes a node, handling cases with no children, one child, or two children, and balances the tree.
- The search() function searches for an element using BST comparisons.
- The inorder() function traverses the tree in the order Left → Root → Right.
- The preorder() function traverses the tree in the order Root → Left → Right.
- The postorder() function traverses the tree in the order Left → Right → Root.
- The level_order() function visits nodes level by level using a queue.
- The display() function displays the AVL tree structure sideways.
- The main menu allows the user to select and perform each AVL tree operation, validate inputs, and exit the program.
- The program demonstrates AVL tree insertion, deletion, searching, balancing, minimum and maximum element finding, height calculation, and traversal methods.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurred when non-integer input was entered.	Used try-except to validate integer inputs.
2.	Duplicate values were entered during insertion.	Checked for duplicate keys before inserting a new node.
3.	Search failed when the given element was not present in the tree.	Checked whether search() returns None before displaying the result.
4.	Minimum or maximum value could not be found when the tree was empty.	Checked whether the root is None before performing the operation.
5.	Deletion failed when the given element was not present in the tree.	Searched for the element before calling the delete() function.
6.	Tree became unbalanced after insertion or deletion.	Applied appropriate LL, RR, LR and RL rotations.
7.	Traversal operations produced no elements when the tree was empty.	Checked whether the root is None before performing traversal.
8.	Incorrect height was obtained after tree modifications.	Updated node heights after insertion, deletion and rotations.

Week-11

AIM: Write a python program to implement Hashing (Seperate chaining, Linear probing, and Quadratic probing).

PROGRAM:

```
#T.Naga Siddhardha
SIZE = 10

# ---------------- SEPARATE CHAINING ----------------

class SeparateChaining:
    def __init__(self):
        self.table = [[] for _ in range(SIZE)]

    def insert(self, key):
        index = key % SIZE
        if key not in self.table[index]:
            self.table[index].append(key)
        print("Inserted successfully.")

    def search(self, key):
        index = key % SIZE
        if key in self.table[index]:
            print("Element found.")
        else:
            print("Element not found.")

    def delete(self, key):
        index = key % SIZE
        if key in self.table[index]:
            self.table[index].remove(key)
            print("Element deleted.")
        else:
            print("Element not found.")

    def display(self):
        for i in range(SIZE):
            print(i, ":", self.table[i])


# ---------------- LINEAR PROBING ----------------

class LinearProbing:
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
            pos = (index + i) % SIZE

            if self.table[pos] == key:
                print("Duplicate element.")
```

```
            if self.table[pos] is None:
                self.table[pos] = key
                print("Inserted successfully.")
                return

        print("Hash table is full.")

    def search(self, key):
        index = key % SIZE

        for i in range(SIZE):
            pos = (index + i) % SIZE

            if self.table[pos] is None:
                break

            if self.table[pos] == key:
                print("Element found.")
                return

        print("Element not found.")

    def delete(self, key):
        index = key % SIZE

        for i in range(SIZE):
            pos = (index + i) % SIZE

            if self.table[pos] is None:
                break

            if self.table[pos] == key:
                self.table[pos] = "DELETED"
                print("Element deleted.")
                return

        print("Element not found.")

    def display(self):
        for i in range(SIZE):
            print(i, ":", self.table[i])


# ---------------- QUADRATIC PROBING ----------------

class QuadraticProbing:
    def __init__(self):
        self.table = [None] * SIZE
```

```
    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
            pos = (index + i * i) % SIZE

            if self.table[pos] == key:
                print("Duplicate element.")
                return

            if self.table[pos] is None or self.table[pos] == "DELETED":
                self.table[pos] = key
                print("Inserted successfully.")
                return

        print("No available position found.")

    def search(self, key):
        index = key % SIZE

        for i in range(SIZE):
            pos = (index + i * i) % SIZE

            if self.table[pos] is None:
                break

            if self.table[pos] == key:
                print("Element found.")
                return

        print("Element not found.")

    def delete(self, key):
        index = key % SIZE

        for i in range(SIZE):
            pos = (index + i * i) % SIZE

            if self.table[pos] is None:
                break

            if self.table[pos] == key:
                self.table[pos] = "DELETED"
                print("Element deleted.")
                return

        print("Element not found.")

    def display(self):
```

```
            print(i, ":", self.table[i])


# ---------------- MAIN PROGRAM ----------------

while True:
    print("\n--- HASHING OPERATIONS ---")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 4:
        print("Exiting...")
        break

    if choice not in [1, 2, 3]:
        print("Invalid choice.")
        continue

    if choice == 1:
        h = SeparateChaining()
    elif choice == 2:
        h = LinearProbing()
    else:
        h = QuadraticProbing()

    while True:
        print("\n1. Insert")
        print("2. Search")
        print("3. Delete")
        print("4. Display")
        print("5. Back to Main Menu")

        op = int(input("Enter operation: "))

        if op == 1:
            key = int(input("Enter key: "))
            h.insert(key)

        elif op == 2:
            key = int(input("Enter key: "))
            h.search(key)

        elif op == 3:
            key = int(input("Enter key: "))
            h.delete(key)
```

OUTPUT:

```
--- HASHING OPERATIONS ---
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 1

1. Insert
2. Search
3. Delete
4. Display
5. Back to Main Menu
Enter operation: 1
Enter key: 2
Inserted successfully.

1. Insert
2. Search
3. Delete
4. Display
5. Back to Main Menu
Enter operation: 1
Enter key: 1
Inserted successfully.

1. Insert
2. Search
3. Delete
4. Display
5. Back to Main Menu
Enter operation: 1
Enter key: 3
Inserted successfully.

1. Insert
2. Search
3. Delete
4. Display
5. Back to Main Menu
Enter operation: 5

--- HASHING OPERATIONS ---
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 2

1. Insert
```

EXPLANATION:

- The size variable defines the size of the hash table as 10.
- The separate_chaining() function creates a hash table with empty lists to store the elements.
- The hash function key % size calculates the index for each key. The append() function inserts keys into the corresponding list. If multiple keys have the same index, they are stored in the same list.
- The linear_probing() function creates an empty hash table using None to represent empty positions.
- The while loop checks whether the calculated index is occupied. If a collision occurs, (index + 1) % size searches for the next available position and inserts the key.

- The quadratic_probing() function resolves collisions by checking alternative positions using square increments.
- The expression (index + i * i) % size calculates the new index. The if condition checks whether the position is empty, and the break statement stops the loop after insertion.
- The i += 1 statement increases the counter to check the next position if a collision occurs.
- The main program accepts multiple integer keys from the user using input (), split (), map (), and list ().
- The three functions are called to perform Separate Chaining, Linear Probing, and Quadratic Probing.
- The program demonstrates hashing, collision detection, and collision resolution using three different techniques.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Collision occurred when multiple keys generated the same index.	Applied separate chaining and probing techniques.
2.	Index error occurred when the table reached its last position.	Used modulo operation to wrap around the table.
3.	Insertion failed when the hash table was full.	Checked for available positions before insertion.
4.	Elements were stored at incorrect positions during linear probing.	Used (index + 1) % size to find the next position.
5.	Elements were stored at incorrect positions during linear probing.	Used (index + 1) % size to find the next position.
6.	Incorrect positions were checked during quadratic probing.	Used the formula (index + i * i) % size.